Supplementary Materials of "On the Reliability of Wahoo Formant Measurements: A Response to Boë et al. (2026)"

Frédéric Berthommier

CNRS, Grenoble INP, GIPSA-Lab, Université Grenoble Alpes,
11 Rue des Mathématiques – BP 46, 38402 Saint Martin d'Hères Cedex, Grenoble, France
frederic.berthommier@gipsa-lab.grenoble-inp.fr

October 2026

International Journal of Primatology (2026), accepted 1 October 2026. All material archived in the repository
`github.com/FBerthommier/baboon-acoustics`
(interactive demos: `FBerthommier.github.io/baboon-acoustics/demos`; Zenodo DOI minted upon first release).

This document describes the methods and the software used to produce every figure of the article and of the present supplement. The analyses are implemented in the documented Python package `baboon_acoustics`: reading a WAV file, performing formant estimation via Parselmouth, applying Iterative Adaptive Inverse Filtering (IAIF) to extract the glottal source and vocal-tract filter, and producing the figures in this supplement are one-liners with the package, as detailed in Section 2. The package is archived at the root of the repository together with its user documentation, guided example, unit tests and interactive demos. The *original* scripts used during the study are archived verbatim in `supplementary/original_scripts/`; Section 4 gives the function-by-function mapping between those frozen scripts and the package, so that every statement of the article can be traced to the exact code that produced it.

Contents

1 Repository overview **1**

2 Methods **2**
2.1 Audio loading and resampling . . . 2
2.2 Formant tracking with Praat's Burg method . . . 2
2.3 LPC and the Levinson–Durbin recursion . . . 2
2.4 Iterative Adaptive Inverse Filtering (IAIF) . . . 2
2.5 Formant extraction from the IAIF envelope . . . 3
2.6 Fundamental frequency estimation . . . 3
2.7 Vocal tract length estimation . . . 4
2.8 Glottal source analysis . . . 4

3 Reproducing the figures **4**

4 Original scripts and provenance **10**

1 Repository overview

The repository is organised as follows:

- `baboon_acoustics/` — the analysis package. One module per concept: `audio_io` (loading, mono conversion, normalisation, resampling), `formants_praat` (Praat/Burg formant tracking), `formants_iaif` (LPC and Iterative Adaptive Inverse Filtering), `f0` (fundamental-frequency estimation), `vtl` (vocal-tract length), `glottal` (glottal source extraction), `plotting` (journal-style figures). Every function carries a complete NumPy-style docstring; the compiled user guide is `docs/baboon-acoustics-documentation.pdf`.
- `scripts/` — thin command-line programs reproducing each figure of the article and of this supplement (Section 3).
- `demos/` — self-contained interactive HTML pages (audio embedded in base64, dynamic plots, playback cursor), readable offline in any browser or online via GitHub Pages.
- `data/` — the audio material: the four wahoo calls of the sequence (`wahoo_1.wav` … `wahoo_4.wav`), the full original recording (`The_Baboon_Bark.wav`), and the reference F0 table (`wahoo_4_f0_filtered_ac.csv`).
- `supplementary/` — the present document, all figures, and the original scripts archived verbatim (`original_scripts/`, Section 4).
- `tests/` — unit tests sanity-checking every analysis step (`python -m unittest discover -s tests`).

The package requires Python 3.9+ with `numpy`, `scipy`, `matplotlib` and `praat-parselmouth` (`pip install -r requirements.txt`); Praat itself need not be installed.

2 Methods

2.1 Audio loading and resampling

```
from baboon_acoustics import load_audio
samples, fs = load_audio("wahoo_4.wav")
```

`load_audio(path, target_fs=44100, normalize=True)` reads a WAV file, converts stereo to mono by taking the first channel, converts samples to floating point, normalises the signal to the range $[-1, 1]$ (optional), and resamples to the target sampling frequency $F_s = 44100\,\text{Hz}$ with a polyphase filter if necessary. All default analysis parameters below are expressed at this rate.

2.2 Formant tracking with Praat's Burg method

```
from baboon_acoustics import extract_praat_formants
times, formants = extract_praat_formants("wahoo_4.wav", n_formants=4)
```

`extract_praat_formants(audio_path, n_formants, formant_ceiling=5500, window=0.040, time_step=0.040, pre_emphasis=50)` runs Praat's *To Formant (burg)* command through the Parselmouth interface with the settings of the article (time step = 40 ms, window length = 40 ms, maximum formant = 5500 Hz, pre-emphasis = 50 Hz). It returns the frame times and a matrix of shape $(n_{\text{formants}}, n_{\text{frames}})$ in Hz, with NaN where Praat found no value. Two helpers support the comparison with fixed-grid analyses: `interpolate_formants()` resamples tracks onto an arbitrary time grid (NaN-aware), and `fill_leading_nan()` replaces the missing first frame (half-filled window) by the first valid value.

2.3 LPC and the Levinson–Durbin recursion

```
from baboon_acoustics import lpc_levinson, lpc_spectrum_db
a, sigma2 = lpc_levinson(frame, order=60)
freqs, env_db = lpc_spectrum_db(a, sigma2, n_fft=4096, fs=44100)
```

`lpc_levinson(x, order)` computes the linear prediction coefficients of a frame using the Levinson–Durbin recursion. The input is first mean-centred; the autocorrelation is estimated via FFT for efficiency. It returns the coefficient vector `a` (with `a[0] = 1`) and the residual variance σ^2; a small regularisation avoids numerical instability. `lpc_spectrum_db()` evaluates the frequency response of the all-pole filter $H(f) = \sqrt{\sigma^2} \,/\, A(f)$ with `scipy.signal.freqz` and returns the frequency axis (Hz) and the magnitude spectrum in dB.

2.4 Iterative Adaptive Inverse Filtering (IAIF)

```
from baboon_acoustics import iaif_frame
Hvt2, sigma2, gn = iaif_frame(frame, fs, vt_order=60)
```

`iaif_frame(frame, fs, vt_order)` performs one frame of the two-iteration IAIF algorithm (Alku, 1991) to separate the glottal source from the vocal-tract filter:

1. Estimate a first glottal filter H_{g1} of order 1 via LPC, and inverse-filter the frame to obtain s_1.
2. Estimate the vocal-tract filter H_{vt1} of order `vt_order` on s_1.
3. Compute a first source estimate $g_1 = \int (H_{vt1} * \text{frame})$ and remove its mean.
4. Estimate a second glottal filter H_{g2} of order 2 on g_1, inverse-filter the original frame to get s_2.
5. Estimate the final vocal-tract filter H_{vt2} (order `vt_order`) on s_2.
6. Compute the final source signal $g_n = \int (H_{vt2} * \text{frame})$ and remove its mean.

The function returns H_{vt2} (LPC coefficients), σ^2 (residual variance), and the glottal source signal g_n.

2.5 Formant extraction from the IAIF envelope

```
from baboon_acoustics import get_frame_formants, iaif_sliding_analysis
```

`get_frame_formants(frame, fs, vt_order=60, n_formants=4, fmax=3500, min_prominence=2.5, min_dist_hz=150)` extracts formant frequencies from a frame by running IAIF, evaluating the vocal-tract envelope, and detecting its spectral peaks below `fmax` with `scipy.signal.find_peaks` (minimum prominence 2.5 dB, minimum spacing 150 Hz). The $n_{\text{formants}} + 2$ most prominent peaks are kept as candidates and the lowest ones in frequency are returned, so that a strong high-frequency peak cannot mask a genuine low formant; missing entries are padded with NaN. This single function replaces the `get_frame_formants` / `get_frame_formants_3` pair of the archived scripts through its `n_formants` parameter.

`iaif_sliding_analysis(x, fs, n_formants=4, ...)` applies the analysis to the whole signal with a sliding Hann window (4096 samples, 75 % overlap). For each frame it computes the spectrogram of the signal, the spectrogram of the glottal source, the envelope $|H_{vt2}(f)|$ interpolated on a common frequency grid, the F0 track (below), and the IAIF formant tracks. The result is a named tuple (`times`, `freqs`, `spectrogram_db`, `glottal_db`, `envelope_db`, `f0_track`, `formants`); it likewise subsumes the `_3` variant of the archived scripts.

2.6 Fundamental frequency estimation

```
from baboon_acoustics import estimate_f0_from_gn, estimate_f0_praat
```

`estimate_f0_from_gn(gn, fs, f0_min=50, f0_max=500, ac_threshold=0.3)` estimates F0 from a glottal source segment by autocorrelation: the mean-centred signal is correlated with itself, the normalised autocorrelation is searched between the lags corresponding to `f0_max` and `f0_min`, and the reciprocal of the best lag is returned; frames whose peak falls below `ac_threshold` (0.3) or touches the search boundaries return NaN.

`estimate_f0_praat(audio_path, use_prefilter=True, ...)` runs Praat's autocorrelation pitch analysis (`to_pitch_ac`; floor 30 Hz, ceiling 800 Hz, time step 0.01 s) and returns the time/F0/strength of the voiced frames. With `use_prefilter=True` the signal first passes through `gaussian_lowpass_filter(samples, fs, cutoff=800, attenuation=0.03)`, a frequency-domain Gaussian window $H(f) = \exp(-(f/\sigma)^2/2)$ with $\sigma = f_{\text{cutoff}}/\sqrt{-2\ln(\alpha)}$ and $\alpha = 0.03$ (-30.5 dB at cutoff). This preprocessing replicates the *filtered autocorrelation* method of Praat 6.4.65 (not available in the legacy Praat embedded in Parselmouth 0.4.5): attenuating the spectrum above the F0 range suppresses the subharmonic candidates responsible for octave-halving errors on calls with nonlinear dynamics (Fig. 3 vs. Fig. 4).

2.7 Vocal tract length estimation

```
from baboon_acoustics import reby_mccomb_vtl
vtl_cm = reby_mccomb_vtl(formants)
```

`reby_mccomb_vtl(formants, c=350.0)` computes a global vocal-tract length estimate (cm) with the Reby–McComb method: every valid (non-NaN) formant measurement contributes one point (order, frequency) with orders $1, 3, 5, 7, \ldots$ for F1, F2, F3, F4, …; a least-squares line through the origin, $F = b{\cdot}\text{order}$, is fitted across all points of all frames ($b = \sum f_i{\cdot}\text{order} \,/\, \sum \text{order}^2$), and VTL $= c/(4b)$ with the speed of sound $c = 350$ m/s. The function accepts any number of formants (the archived scripts' `_3` and four-formant variants are subsumed) and returns NaN when fewer than two valid values are available.

2.8 Glottal source analysis

```
from baboon_acoustics import (extract_glottal_pulse,
                              extract_glottal_train)
t_pulse, pulse = extract_glottal_pulse(x, fs, 0.26, 0.34)
t_train, train = extract_glottal_train(x, fs)
```

`extract_glottal_pulse(x, fs, t_start, t_end)` isolates a single glottal cycle from a stable segment: the segment (Hamming windowed) is decomposed by IAIF, the fundamental period is measured from the autocorrelation of the source, and one period is cut around the largest absolute excursion (Fig. 2).

`extract_glottal_train(x, fs)` reconstructs the full pulse train by overlap-add of Hann-windowed IAIF estimates (window 4096 samples, hop 1024), trimmed at the edges where too few windows overlap.

`normalize_amplitude(signal, fs, cutoff=50)` removes slow drifts (floating DC) by subtracting a low-pass filtered version of the signal and divides by a low-pass filtered envelope; it is applied before the phase portraits of Figs. 5 and 6, where amplitude modulation would otherwise blur the trajectories. `plot_phase_portrait(ax, signal, fs)` draws the Hilbert-plane portrait of the analytic signal.

3 Reproducing the figures

Each figure is reproduced by one thin command-line script of the `scripts/` folder, which only orchestrates calls to the package:

```
python scripts/make_figure1.py # article Fig. 1
python scripts/make_figure_multipanel.py # Fig. 1 + Fig. 2 of this supplement
python scripts/make_figure_ac.py # Fig. 3 (+ CSV table)
python scripts/make_figure_glottal.py # Figs. 5 and 6 (--segment wa|hoo)
python scripts/make_figure_f0.py # F0 contour used in the article
```

All scripts default to `data/wahoo_4.wav` — the last call of the sequence originally recorded in `The_Baboon_Bark.wav` and analysed by Boë et al. (2026); the three other calls of the sequence can be processed by passing their filenames as an argument. The article's Figure 1 compares the Praat/Burg tracks obtained with $N = 4$ and $N = 6$ poles, the merged trajectory using the optimal N per segment (first frame from $N = 6$, frames 2–9 from $N = 4$, remaining frames from $N = 6$) against the reference data digitized from Boë et al. (2026, their Fig. 10h), with one Reby–McComb VTL estimate per panel.

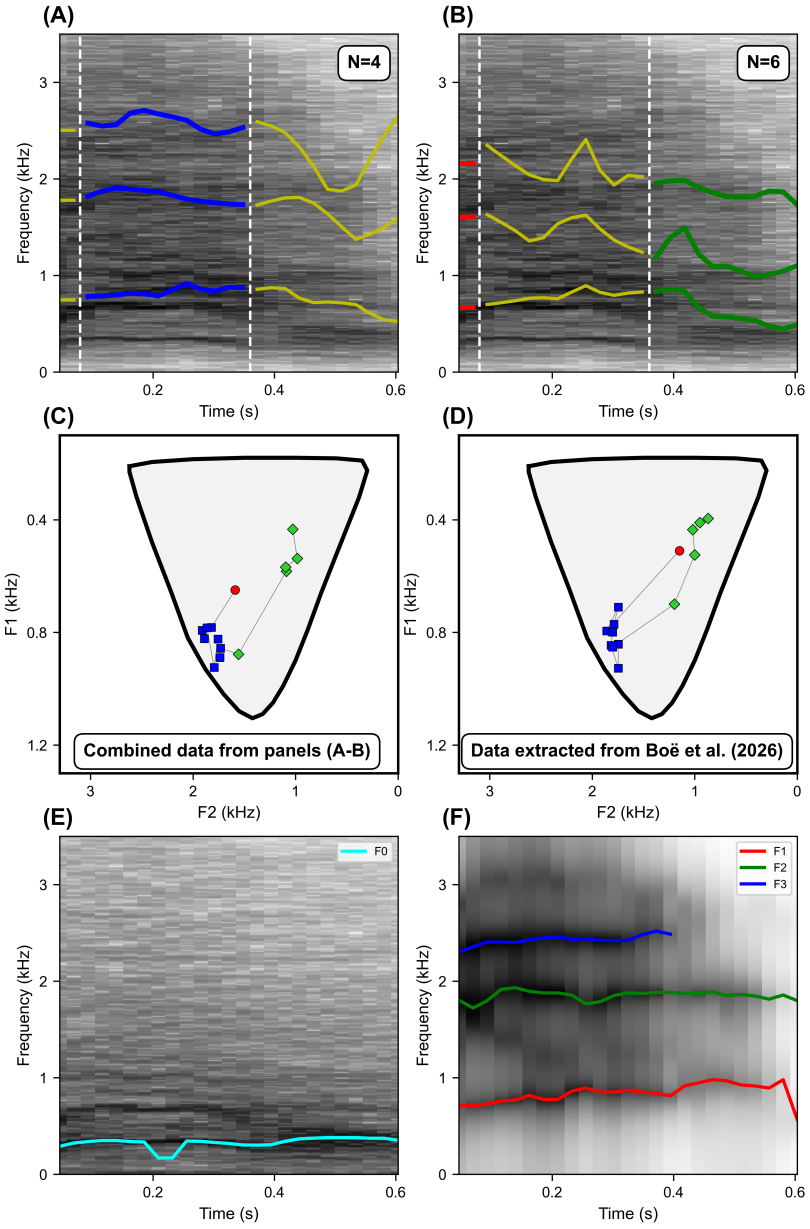


Figure 1: Multi-panel analysis of a baboon wahoo call (exemplar file `wahoo_4.wav`), produced by `scripts/make_figure_multipanel.py`. (A) Wideband spectrogram of the original signal (Hann window, length = 4096 samples, hop = 1024 samples, 44.1 kHz sampling rate). Overlaid formant trajectories F1–F3 (yellow before 0.08 s, blue between 0.08 and 0.36 s, yellow again after 0.36 s) were extracted using Praat's Burg algorithm via Parselmouth (*To Formant (burg)*) with parameters: time step = 40 ms, window length = 40 ms, maximum formant = 5500 Hz, pre-emphasis = 50 Hz, number of formants = 4. (B) Same as (A) but with number of formants = 6; formant colours are now red for the first segment, yellow for the middle, green for the last segment. (C) Merged F1–F2 trajectories combining the first point from the $N = 6$ extraction (red circle) and the subsequent points from the $N = 4$ extraction (blue squares and green diamonds). The grey shaded area and black line represent the maximal acoustic space (MAS) contour approximated from Boë et al. (2026). (D) F1–F2 trajectories manually extracted from the spectrogram published in Boë et al. (2026) (red circle, blue squares, green diamonds), superimposed on the same MAS contour. (E) Spectrogram of the glottal source estimate obtained after the second iteration of IAIF (LPC order = 60). The cyan line shows the fundamental frequency (F0) estimated from the source signal by autocorrelation (search range 50–500 Hz, voicing threshold = 0.3). (F) Spectrogram of the vocal-tract filter envelope $|H_{vt2}(f)|$ computed from the IAIF filter coefficients (frame-wise LPC, order = 60, same framing as in A and B). Overlaid formant tracks (F1 red, F2 green, F3 blue) are IAIF-based estimates obtained by peak picking on the envelope (minimum prominence = 2.5 dB, minimum peak distance = 150 Hz). All spectrograms use a grey (*Greys*) colour map. Dashed white vertical lines at 0.08 s and 0.36 s delimit the [w], [æ] and [u] segments.

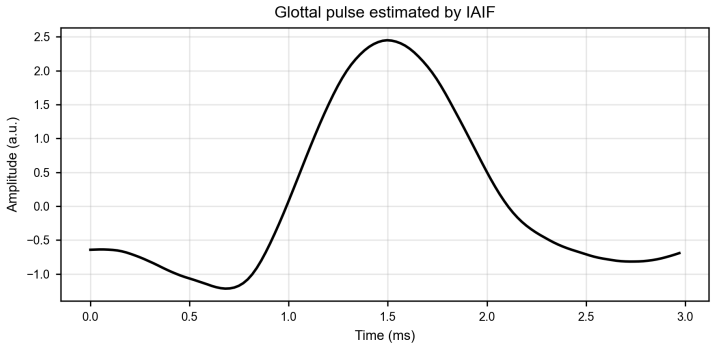


Figure 2: Single glottal pulse estimated by IAIF on the segment 260–340 ms of `wahoo_4.wav` (`extract_glottal_pulse`; Hamming window, autocorrelation period, extraction around the largest absolute excursion of the source signal).

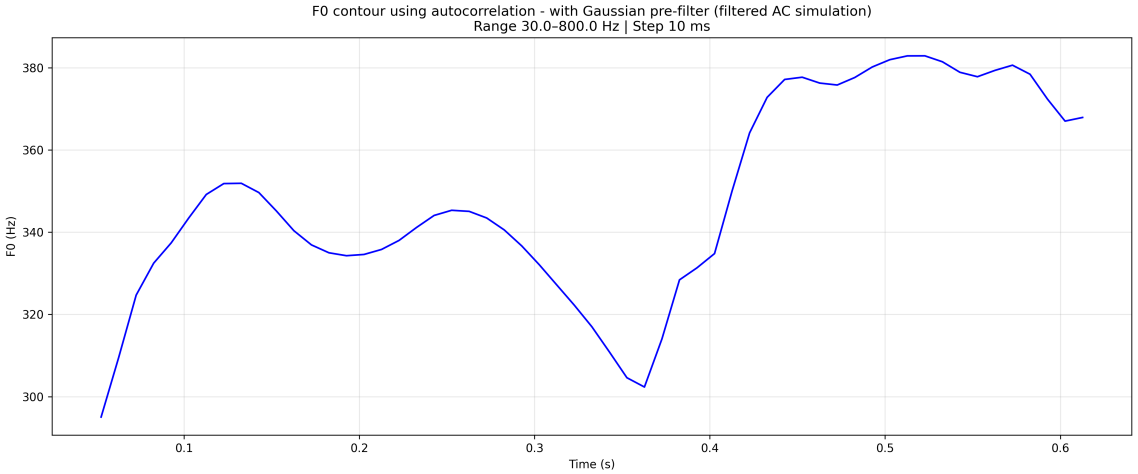


Figure 3: F0 contour obtained after Gaussian low-pass pre-filtering (cutoff 800 Hz, attenuation −30.5 dB at cutoff) followed by Praat's autocorrelation pitch algorithm (*to_pitch_ac*) with floor 30 Hz, ceiling 800 Hz, and time step 0.01 s. The analysis yielded 57 voiced frames, a mean F0 of 348.2 Hz (standard deviation 24.3 Hz). This preprocessing effectively suppresses subharmonics and produces a smooth pitch track comparable to Praat 6.4.65's *To Pitch (filtered ac)…* command.

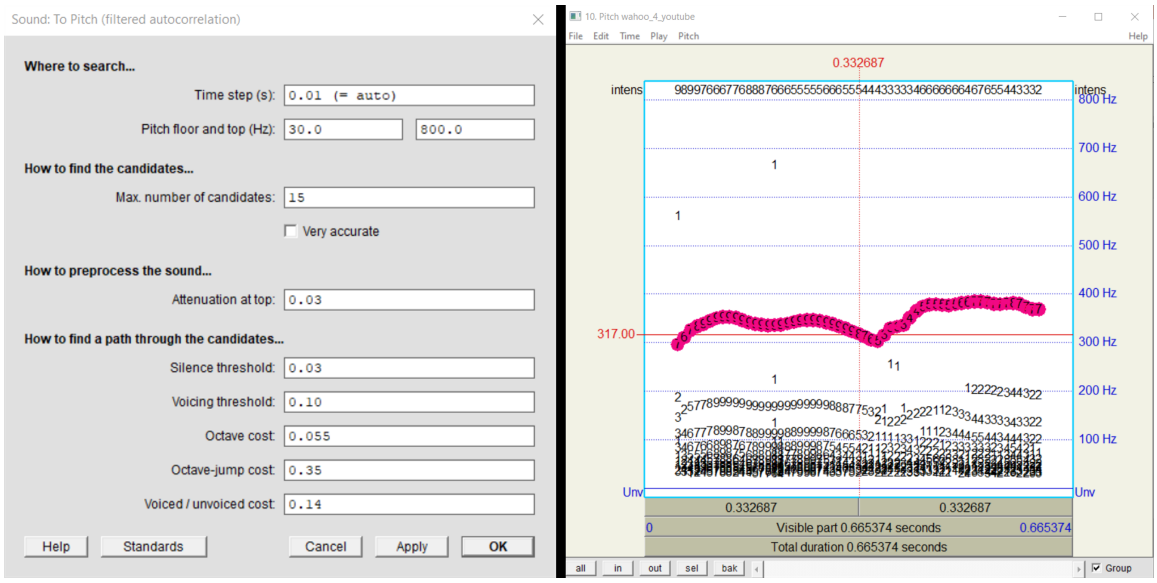


Figure 4: Left: settings of Praat 6.4.65's *To Pitch (filtered autocorrelation)* command. Right: resulting pitch contour of `wahoo_4.wav` displayed in Praat (mean 317 Hz over the visible part). The Python pre-filtering replicates this behaviour.

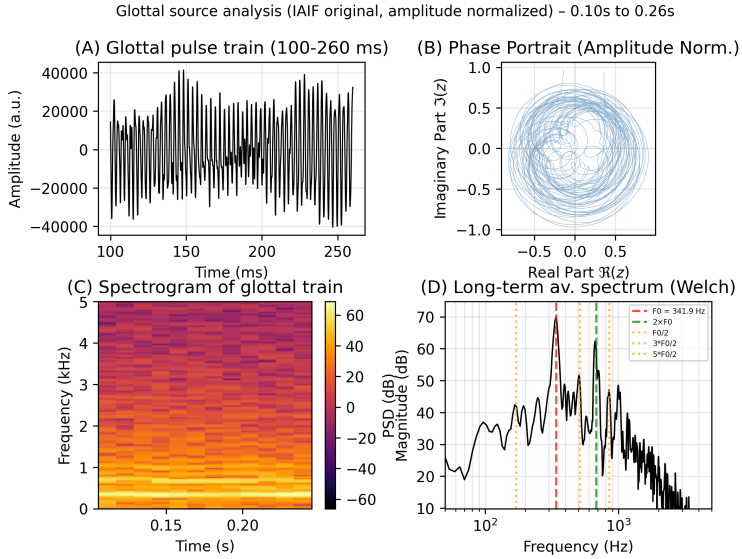


Figure 5: Glottal source analysis (IAIF) — "wa" segment (100–260 ms), produced by `scripts/make_figure_glottal.py -segment wa`. (A) Glottal pulse train: the amplitude does not systematically alternate between successive cycles, preventing a definitive conclusion of period doubling. (B) Phase portrait (Hilbert plane): the trajectory forms a thickened annular band rather than a single closed loop. (C) Spectrogram: while a fundamental harmonic and possibly some sub-harmonics are present in the source, the two-frequency dynamics is not visibly apparent. (D) Long-term average spectrum (Welch): in addition to the dominant peak at $F_0 = 341.9$ Hz, spectral ridges appear at $F_0/2$, $3F_0/2$, and $5F_0/2$ (orange dotted lines), though their energy is substantially lower than that of the integer harmonics. *Conclusion:* taken together, these features indicate that the glottal source deviates from a purely harmonic and decreasing (e.g., -12 dB/octave) spectral model, challenging the linear source–filter separation assumption for this segment.

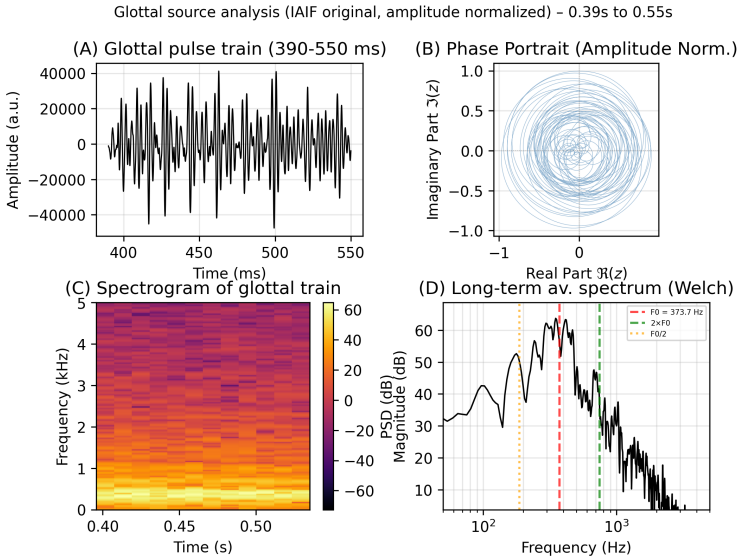


Figure 6: Glottal source analysis (IAIF) — "hoo" segment (390–550 ms), produced by `scripts/make_figure_glottal.py` (default segment). (A) Glottal pulse train: cycle-to-cycle amplitude and timing are strongly irregular with no discernible repeating pattern, indicating a breakdown of any low-period orbital structure present in the earlier segment. (B) Phase portrait (Hilbert plane): the trajectory fills a broad, irregular disk extending beyond unit radius, with no coherent winding structure; this irregular geometry evokes a chaotic-like dynamical regime, although a more precise characterization would require additional analysis. (C) Spectrogram: a broad and thick spectral peak is observed, with a weak harmonic structure, superimposed on a continuous noise floor. (D) Long-term average spectrum (Welch): the fundamental peak is substantially broadened, while a prominent subharmonic peak appears near $F_0/2$ and weak harmonic peaks near $2F_0$; a continuous spectral background is visible between harmonic ridges. *Conclusion:* taken together, these observations indicate a marked departure from a regular periodic source, where the aperiodicity is sufficiently severe to challenge the linear source–filter separation, and therefore any LPC-based formant estimate, for this segment.

4 Original scripts and provenance

The scripts actually run for the study are archived *verbatim* in `supplementary/original_scripts/` (`make_figure_1.py`, `make_figure_multipanel.py`, `make_figure_ac.py`, `make_figure_glottal.py`). They are frozen for provenance; new work should use the package, which implements the same algorithms. Table 1 maps each function documented in earlier versions of this supplement to its package equivalent.

Archived script function	Package equivalent
`load_audio`	`audio_io.load_audio`
`lpc_levinson`	`formants_iaif.lpc_levinson`
`lpc_spectrum_db`	`formants_iaif.lpc_spectrum_db`
`iaif_frame`	`formants_iaif.iaif_frame`
`get_frame_formants`, `get_frame_formants_3`	`formants_iaif.get_frame_formants` (parameter `n_formants`)
`iaif_sliding_analysis`, `iaif_sliding_analysis_3`	`formants_iaif.iaif_sliding_analysis` (parameter `n_formants`)
`estimate_f0_from_gn`	`f0.estimate_f0_from_gn`
`gaussian_lowpass_filter`, `analyze_pitch_ac`	`f0.gaussian_lowpass_filter`, `f0.estimate_f0_praat`
`reby_mccomb_vtl_global`, `reby_mccomb_vtl_global_3`	`vtl.reby_mccomb_vtl`
`extract_parselmouth_formants_n`	`formants_praat.extract_praat_formants` with `formants_praat.interpolate_formants`
`extract_and_save_glottal_pulse`	`glottal.extract_glottal_pulse`
`extract_full_glottal_train`	`glottal.extract_glottal_train`
`normalize_amplitude`	`glottal.normalize_amplitude`
`plot_f1f2_trajectory`, `setup_frequency_axis`, `add_panel_label`	`plotting.plot_f1f2_trajectory`, `plotting.setup_frequency_axis`, `plotting.add_panel_label`
`fix_initial_nan`	`formants_praat.fill_leading_nan`
`plot_phase_portrait`	`plotting.plot_phase_portrait`

Table 1: Mapping between the functions of the archived original scripts and the `baboon_acoustics` package. Where two variants existed (three vs. four formants), a single parameterised function now covers both.

Reference

Boë, L.-J., Laporte, M., Sawallis, T. R., Badin, P., & Schwartz, J.-L. (2026). Stalking the wild formant: How to reliably measure formants in primate vocalizations. *International Journal of Primatology*, 47(3), 799–841. https://doi.org/10.1007/s10764-026-00551-4